

LAPORAN PRAKTIKUM STRUKTUR DATA

“Implementasi Algoritma Shell Sort, Quick Sort, dan Merge Sort pada Java”

disusun Oleh:

Az Zahrand Solichul Tajussalathin

NIM 2511532001

Dosen Pengampu: Dr. Wahyudi S.T M.T

Asisten Pratikum: Jovantri Immanuel Gulo



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

DAFTAR ISI

DAFTAR ISI	i
KATA PENGANTAR.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum	1
BAB II PEMBAHASAN	2
2.1 Program shellSort	2
2.2 Program QuickSort.....	3
2.3 Program MergeSort	5
2.4 Program BubbleSortGUI	6
2.4 Program MergeSortGUI	11
BAB III TUGAS	18
3.1 Program Sorting.....	18
3.2 Alasan Implementasi Algoritma	22
BAB IV KESIMPULAN	24
4.1 Ringkasan Hasil Praktikum	24
DAFTAR PUSTAKA.....	iii

KATA PENGANTAR

Puji syukur kehadiran Allah SWT atas limpahan rahmat dan karunia-Nya sehingga laporan praktikum dengan judul “Implementasi Algoritma Shell Sort, Quick Sort, dan Merge Sort pada Java” ini dapat disusun dan diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk pelaksanaan kegiatan praktikum pada mata kuliah Struktur Data.

Ucapan terima kasih disampaikan kepada Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu, serta kepada asisten laboratorium yang telah memberikan bimbingan selama kegiatan praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi penulisan maupun isi. Oleh karena itu, saran dan kritik yang bersifat membangun sangat diharapkan demi perbaikan di masa yang akan datang. Harapannya, laporan ini dapat memberikan manfaat dan menjadi referensi bagi pembelajaran dalam bahasa pemrograman Java.

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Pada struktur data berskala kecil, algoritma pengurutan dasar seperti Bubble Sort atau Insertion Sort masih dapat diandalkan. Namun, ketika berhadapan dengan dataset berskala masif, algoritma tersebut akan membebani kinerja memori dan prosesor secara drastis. Oleh karena itu, ilmu komputer mengembangkan algoritma Advanced Sorting. Algoritma seperti Merge Sort dan Quick Sort memanfaatkan paradigma Divide and Conquer sedangkan Shell Sort memodifikasi perbandingan elemen dengan gap tertentu.

1.2 Tujuan Praktikum

Tujuan dari praktikum ini adalah:

1. Memahami konsep dan implementasi algoritma pengurutan tingkat lanjut, yaitu Shell Sort, Quick Sort, dan Merge Sort.
2. Mempelajari dan menerapkan teknik Divide and Conquer pada algoritma Merge Sort dan Quick Sort.
3. Menganalisis optimalisasi pemilihan pivot pada Quick Sort menggunakan metode Median-of-Three.

1.3 Manfaat Praktikum

Manfaat yang diperoleh dari praktikum ini antara lain:

1. Mengasah kemampuan analisis komparatif dalam memilih algoritma pengurutan yang tepat sesuai dengan arsitektur memori program.
2. Meningkatkan kemampuan logika dalam melacak alur eksekusi fungsi rekursif yang digunakan pada Merge Sort dan Quick Sort

BAB 2

PEMBAHASAN

2.1 Program InsertionSort

```
package pekan8_2511532001;

public class shellSort_2511532001 {
    public static void shellSort_2001(int[] A_2001) {
        int n_2001 = A_2001.length;
        int gap_2001 = n_2001 / 2;

        while (gap_2001 > 0) {
            for (int i_2001 = gap_2001; i_2001 < n_2001; i_2001++) {
                int temp_2001 = A_2001[i_2001];
                int j_2001 = i_2001;

                while (j_2001 >= gap_2001 && A_2001[j_2001 - gap_2001] >
temp_2001) {
                    A_2001[j_2001] = A_2001[j_2001 - gap_2001];
                    j_2001 = j_2001 - gap_2001;
                }

                A_2001[j_2001] = temp_2001;
            }

            gap_2001 = gap_2001 / 2;
        }
    }

    public static void main(String[] args) {
        int[] data_2001 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};

        System.out.print("Sebelum: ");
        printArray_2001(data_2001);

        shellSort_2001(data_2001);

        System.out.print("Sesudah (Shell Sort): ");
        printArray_2001(data_2001);
    }

    public static void printArray_2001(int[] arr_2001) {
        for (int i_2001 : arr_2001) {
            System.out.print(i_2001 + " ");
        }
        System.out.println();
    }
}
```

Program shellSort merupakan optimasi langsung dari Insertion Sort. Algoritma ini membandingkan elemen yang dipisahkan oleh gap. Pada program ini, gap awal ditentukan dari setengah panjang array ($n / 2$), kemudian terus dibagi dua pada setiap iterasinya hingga gap bernilai 1. Metode ini memungkinkan elemen yang bernilai sangat kecil di ujung kanan array untuk melompat jauh ke kiri dengan cepat, menutupi kelemahan utama Insertion Sort yang harus menggeser elemen satu per satu.

2.2 Program QuickSort

```
package pekan8_2511532001;

public class QuickSort_2511532001 {
    static void swap_2001(int[] arr_2001, int i_2001, int j_2001) {
        int temp_2001 = arr_2001[i_2001];
        arr_2001[i_2001] = arr_2001[j_2001];
        arr_2001[j_2001] = temp_2001;
    }

    // Metode tambahan untuk mengatur pivot menggunakan Median-of-Three
    static void medianOfThree_2001(int[] arr_2001, int low_2001, int high_2001) {
        int mid_2001 = low_2001 + (high_2001 - low_2001) / 2;

        // Urutkan elemen low, mid, dan high
        if (arr_2001[low_2001] > arr_2001[mid_2001]) {
            swap_2001(arr_2001, low_2001, mid_2001);
        }
        if (arr_2001[low_2001] > arr_2001[high_2001]) {
            swap_2001(arr_2001, low_2001, high_2001);
        }
        if (arr_2001[mid_2001] > arr_2001[high_2001]) {
            swap_2001(arr_2001, mid_2001, high_2001);
        }
        swap_2001(arr_2001, mid_2001, high_2001);
    }

    static int partition_2001(int[] arr_2001, int low_2001, int high_2001) {
        // Panggil fungsi medianOfThree sebelum menentukan pivot
        medianOfThree_2001(arr_2001, low_2001, high_2001);

        int pivot_2001 = arr_2001[high_2001]; // Sekarang arr[high] sudah berisi nilai median
        int i_2001 = (low_2001 - 1);

        for (int j_2001 = low_2001; j_2001 <= high_2001 - 1; j_2001++) {
            // Jika elemen saat ini lebih kecil dari atau sama dengan pivot
            if (arr_2001[j_2001] < pivot_2001) {
                // Increment indeks elemen yang lebih kecil
            }
        }
    }
}
```

```

        i_2001++;
        swap_2001(arr_2001, i_2001, j_2001);
    }
}
swap_2001(arr_2001, i_2001 + 1, high_2001);
return (i_2001 + 1);
}
static void quickSort_2001(int[] arr_2001, int low_2001, int high_2001)
{
    if (low_2001 < high_2001) {
        int pi_2001 = partition_2001(arr_2001, low_2001, high_2001);

        quickSort_2001(arr_2001, low_2001, pi_2001 - 1);
        quickSort_2001(arr_2001, pi_2001 + 1, high_2001);
    }
}

public static void printArr_2001(int[] arr_2001) {
    for (int i_2001 = 0; i_2001 < arr_2001.length; i_2001++) {
        System.out.print(arr_2001[i_2001] + " ");
    }
    System.out.println();
}

public static void main(String[] args) {
    int[] arr_2001 = { 10, 7, 8, 9, 1, 5 };
    int N_2001 = arr_2001.length;

    System.out.print("Data sebelum diurutkan: ");
    printArr_2001(arr_2001);

    quickSort_2001(arr_2001, 0, N_2001 - 1);

    System.out.print("Data Terurut QuickSort: ");
    printArr_2001(arr_2001);
}
}

```

Program QuickSort menerapkan paradigma Divide and Conquer. Keunggulan utama pada implementasi ini adalah penggunaan metode medianOfThree untuk menentukan pivot. Alih-alih mengambil elemen terakhir secara buta (yang berisiko memicu performa terburuk jika data sudah terurut), program membandingkan nilai awal, tengah, dan akhir array, lalu mengambil nilai tengahnya sebagai pivot. Setelah pivot ditentukan, metode partition akan menata ulang elemen sehingga semua nilai yang lebih kecil dari pivot berada di sebelah kiri, dan yang lebih besar di sebelah kanan. Proses ini diulang secara rekursif (quickSort) pada sub-array kiri dan kanan hingga seluruh data terurut.

2.3 Program MergeSort

```
package pekan8_2511532001;

public class MergeSort_2511532001 {
    void merge_2001(int arr_2001[], int l_2001, int m_2001, int r_2001) {
        // Find sizes of two subarrays to be merged
        int n1_2001 = m_2001 - l_2001 + 1;
        int n2_2001 = r_2001 - m_2001;

        /* Create temp arrays */
        int L_2001[] = new int[n1_2001];
        int R_2001[] = new int[n2_2001];

        /* Copy data to temp arrays */
        for (int i_2001 = 0; i_2001 < n1_2001; ++i_2001)
            L_2001[i_2001] = arr_2001[l_2001 + i_2001];

        for (int j_2001 = 0; j_2001 < n2_2001; ++j_2001)
            R_2001[j_2001] = arr_2001[m_2001 + 1 + j_2001];

        int i_2001 = 0, j_2001 = 0;

        // Initial index of merged subarray array
        int k_2001 = l_2001;
        while (i_2001 < n1_2001 && j_2001 < n2_2001) {
            if (L_2001[i_2001] <= R_2001[j_2001]) {
                arr_2001[k_2001] = L_2001[i_2001];
                i_2001++;
            } else {
                arr_2001[k_2001] = R_2001[j_2001];
                j_2001++;
            }
            k_2001++;
        }

        /* Copy remaining elements of L[] if any */
        while (i_2001 < n1_2001) {
            arr_2001[k_2001] = L_2001[i_2001];
            i_2001++;
            k_2001++;
        }

        /* Copy remaining elements of R[] if any */
        while (j_2001 < n2_2001) {
            arr_2001[k_2001] = R_2001[j_2001];
            j_2001++;
            k_2001++;
        }
    }

    void sort_2001(int arr_2001[], int l_2001, int r_2001) {
        if (l_2001 < r_2001) {

```



```

        // find
        int m_2001 = l_2001 + (r_2001 - l_2001) / 2;

        // sort
        sort_2001(arr_2001, l_2001, m_2001);
        sort_2001(arr_2001, m_2001 + 1, r_2001);

        // merge
        merge_2001(arr_2001, l_2001, m_2001, r_2001);
    }
}

static void printArray_2001(int arr[]) {
    int n = arr.length;
    for (int i = 0; i < n; ++i) {
        System.out.print(arr[i] + " ");
    }
    System.out.println();
}

public static void main(String args[]) {
    int arr[] = { 12, 11, 13, 5, 6, 7};
    System.out.println("Sebelum terurut");
    printArray_2001(arr);
    MergeSort_2511532001 ob = new MergeSort_2511532001();
    ob.sort_2001(arr, 0, arr.length - 1);
    System.out.println("\nSesudah terurut menggunakan merge sort");
    printArray_2001(arr);
}
}

```

Program MergeSort juga berbasis Divide and Conquer. Namun, pendekatannya berbeda: program akan membelah array tepat di tengah (m) secara terus-menerus melalui fungsi sort hingga tersisa elemen-elemen tunggal. Setelah terpecah, fungsi merge mengambil alih untuk menyatukan kembali pecahan-pecahan tersebut secara terurut. Fungsi ini bekerja dengan menyalin pecahan ke dalam array sementara (L dan R), lalu membandingkan indeks terdepan dari kedua array tersebut untuk dimasukkan kembali ke array utama.

2.4 Program BubbleSortGUI

```

package pekan8_2511532001;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.EventQueue;
import java.awt.FlowLayout;
import java.awt.Font;

```

```

import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;
import javax.swing.JTextArea;
import javax.swing.border.EmptyBorder;

public class BubbleSortGUI_2511532001 extends JFrame {

    private static final long serialVersionUID_2001 = 1L;
    private int[] array_2001;
    private JLabel[] labelArray_2001;
    private JButton stepButton_2001, resetButton_2001, setButton_2001;
    private JTextField inputField_2001;
    private JPanel panelArray_2001;
    private JTextArea stepArea_2001;

    private int i_2001 = 1, j_2001;
    private boolean sorting_2001 = false;
    private int stepCount_2001 = 1;

    private JPanel contentPane_2001;

    /**
     * Create the frame.
     */
    public BubbleSortGUI_2511532001() {
        setTitle("Insertion Sort Langkah per Langkah");
        setSize(750, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        // Panel input
        JPanel inputPanel_2001 = new JPanel(new FlowLayout());
        inputField_2001 = new JTextField(30);
        setButton_2001 = new JButton("Set Array");
        inputPanel_2001.add(new JLabel("Masukkan angka (pisahkan dengan koma):"));
        inputPanel_2001.add(inputField_2001);
        inputPanel_2001.add(setButton_2001);

        // Panel array visual
        panelArray_2001 = new JPanel();
        panelArray_2001.setLayout(new FlowLayout());

        // Panel kontrol

```

```

        JPanel controlPanel_2001 = new JPanel();
        stepButton_2001 = new JButton("Langkah Selanjutnya");
        resetButton_2001 = new JButton("Reset");
        stepButton_2001.setEnabled(false);
        controlPanel_2001.add(stepButton_2001);
        controlPanel_2001.add(resetButton_2001);

        // Area teks untuk log langkah-langkah
        stepArea_2001 = new JTextArea(8, 60);
        stepArea_2001.setEditable(false);
        stepArea_2001.setFont(new Font("Monospaced", Font.PLAIN, 14));
        JScrollPane scrollPane_2001 = new JScrollPane(stepArea_2001);

        // Tambahkan panel ke frame
        add(inputPanel_2001, BorderLayout.NORTH);
        add(panelArray_2001, BorderLayout.CENTER);
        add(controlPanel_2001, BorderLayout.SOUTH);
        add(scrollPane_2001, BorderLayout.EAST);

        // Event Set Array
        stepButton_2001.addActionListener(e_2001 ->
setArrayFromInput_2001());

        // Event Langkah Selanjutnya
        stepButton_2001.addActionListener(e_2001 -> performStep_2001());

        // Event Reset
        resetButton_2001.addActionListener(e_2001 -> reset_2001());
    }
    private void setArrayFromInput_2001() {
        String text_2001 = inputField_2001.getText().trim();
        if (text_2001.isEmpty()) return;
        String[] parts_2001 = text_2001.split(",");
        array_2001 = new int[parts_2001.length];

        try {
            for (int k_2001 = 0; k_2001 < parts_2001.length; k_2001++) {
                array_2001[k_2001] =
Integer.parseInt(parts_2001[k_2001].trim());
            }
        } catch (NumberFormatException e_2001) {
            JOptionPane.showMessageDialog(this, "Masukkan hanya angka "
                + "yang dipisahkan koma!", "Error",
JOptionPane.ERROR_MESSAGE);
            return;
        }

        i_2001 = 0;
        j_2001 = 0;
        stepCount_2001 = 1;
        sorting_2001 = true;
        stepButton_2001.setEnabled(true);
        stepArea_2001.setText("");
    }

```

```

        panelArray_2001.removeAll();
        labelArray_2001 = new JLabel[array_2001.length];

        for (int k_2001 = 0; k_2001 < array_2001.length; k_2001++) {
            labelArray_2001[k_2001] = new
JLabel(String.valueOf(array_2001[k_2001]));
            labelArray_2001[k_2001].setFont(new Font("Arial", Font.BOLD,
24));

            labelArray_2001[k_2001].setOpaque(true);
            labelArray_2001[k_2001].setBackground(Color.WHITE);

labelArray_2001[k_2001].setBorder(BorderFactory.createLineBorder(Color.BLACK));
            labelArray_2001[k_2001].setPreferredSize(new Dimension(50,
50));

labelArray_2001[k_2001].setHorizontalAlignment(SwingConstants.CENTER);
            panelArray_2001.add(labelArray_2001[k_2001]);
        }

        panelArray_2001.revalidate();
        panelArray_2001.repaint();
    }

    private void performStep_2001() {
        if (!sorting_2001 || i_2001 >= array_2001.length - 1) {
            sorting_2001 = false;
            stepButton_2001.setEnabled(false);
            JOptionPane.showMessageDialog(this, "Sorting selesai!");
            return;
        }

        resetHighlights_2001();
        StringBuilder stepLog_2001 = new StringBuilder();

        labelArray_2001[j_2001].setBackground(Color.CYAN);
        labelArray_2001[j_2001 + 1].setBackground(Color.CYAN);

        if (array_2001[j_2001] > array_2001[j_2001 + 1]) {
            // Swap
            int temp_2001 = array_2001[j_2001];
            array_2001[j_2001] = array_2001[j_2001 + 1];
            array_2001[j_2001 + 1] = temp_2001;

            labelArray_2001[j_2001].setBackground(Color.RED);
            labelArray_2001[j_2001 + 1].setBackground(Color.RED);

            stepLog_2001.append("Langkah
").append(stepCount_2001).append(": Menukar elemen ke-")
                .append(j_2001).append(" ").append(array_2001[j_2001 +
1]).append(")")
                .append(" dengan ke-")
                .append(j_2001 + 1).append("
").append(array_2001[j_2001]).append(")\n");

```

```

    } else {
        stepLog_2001.append("Langkah ").append(stepCount_2001)
            .append(": Tidak ada pertukaran antara ke-")
            .append(j_2001).append(" dan ke-").append(j_2001 +
1).append("\n");
    }

    stepLog_2001.append("Hasil:
").append(arrayToString_2001(array_2001)).append("\n\n");
    stepArea_2001.append(stepLog_2001.toString());

    updateLabels_2001();

    j_2001++;
    if (j_2001 >= array_2001.length - i_2001 - 1) {
        j_2001 = 0;
        i_2001++;
    }

    stepCount_2001++;

    if (i_2001 >= array_2001.length - 1) {
        sorting_2001 = false;
        stepButton_2001.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Sorting selesai!");
    }
}
private void updateLabels_2001() {
    for (int k_2001 = 0; k_2001 < array_2001.length; k_2001++) {
labelArray_2001[k_2001].setText(String.valueOf(array_2001[k_2001]));
    }
}

private void resetHighlights_2001() {
    for (JLabel label_2001 : labelArray_2001) {
        label_2001.setBackground(Color.WHITE);
    }
}

private void reset_2001() {
    inputField_2001.setText("");
    panelArray_2001.removeAll();
    panelArray_2001.revalidate();
    panelArray_2001.repaint();
    stepArea_2001.setText("");
    stepButton_2001.setEnabled(false);
    sorting_2001 = false;
    i_2001 = 0;
    j_2001 = 0;
    stepCount_2001 = 1;
}

```

```

    private String arrayToString_2001(int[] arr_2001) {
        StringBuilder sb_2001 = new StringBuilder();
        for (int k_2001 = 0; k_2001 < arr_2001.length; k_2001++) {
            sb_2001.append(arr_2001[k_2001]);
            if (k_2001 < arr_2001.length - 1) sb_2001.append(", ");
        }
        return sb_2001.toString();
    }
}

```

Program BubbleSortGUI menggunakan kerangka GUI yang identik dengan program visualisasi pengurutan sebelumnya. Perbedaannya terletak pada implementasi logika di dalam method performStep. Metode ini mengeksekusi algoritma Bubble Sort tahap demi tahap dengan memanfaatkan variabel global i dan j untuk melacak posisi indeks iterasi saat tombol ditekan. Secara visual, elemen yang sedang dibandingkan (j dan j+1) akan disorot dengan warna tertentu (seperti Cyan). Apabila kondisi $arr[j] > arr[j+1]$ terpenuhi, program akan menukar posisi (swap) elemen di dalam array serta memperbarui label pada antarmuka grafis dengan sorotan warna merah (Red) sebagai penanda terjadinya pertukaran data secara real-time.

2.4 Program MergeSortGUI

```

package pekan8_2511532001;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;

import java.util.LinkedList;
import java.util.Queue;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;

public class MergeSortGUI_2511532001 extends JFrame {

```

```

private static final long serialVersionUID 2001 = 1L;

private int[] array_2001;
private JLabel[] labelArray_2001;
private JButton stepButton_2001, resetButton_2001, setButton_2001;
private JTextField inputField_2001;
private JPanel panelArray_2001;
private JTextArea stepArea_2001;

private int i_2001, j_2001, k_2001;
private int left_2001, mid_2001, right_2001;
private int[] temp_2001;

private boolean isMerging_2001 = false;
private boolean copying_2001 = false;
private int stepCount_2001 = 1;

private Queue<int[]> mergeQueue_2001 = new LinkedList<>();

public MergeSortGUI_2511532001() {
    setTitle("Merge Sort Langkah per Langkah");
    setSize(750, 400);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout(new BorderLayout());

    // Panel input
    JPanel inputPanel_2001 = new JPanel(new FlowLayout());
    inputField_2001 = new JTextField(30);
    setButton_2001 = new JButton("Set Array");
    inputPanel_2001.add(new JLabel("Masukkan angka (pisahkan dengan
koma):"));
    inputPanel_2001.add(inputField_2001);
    inputPanel_2001.add(setButton_2001);

    // Panel array visual
    panelArray_2001 = new JPanel();
    panelArray_2001.setLayout(new FlowLayout());

    // Panel kontrol
    JPanel controlPanel_2001 = new JPanel();
    stepButton_2001 = new JButton("Langkah Selanjutnya");
    resetButton_2001 = new JButton("Reset");
    stepButton_2001.setEnabled(false);
    controlPanel_2001.add(stepButton_2001);
    controlPanel_2001.add(resetButton_2001);

    // Area teks untuk log langkah-langkah
    stepArea_2001 = new JTextArea(8, 60);
    stepArea_2001.setEditable(false);
    stepArea_2001.setFont(new Font("Monospaced", Font.PLAIN, 14));
    JScrollPane scrollPane_2001 = new JScrollPane(stepArea_2001);

```

```

// Tambahkan panel ke frame
add(inputPanel_2001, BorderLayout.NORTH);
add(panelArray_2001, BorderLayout.CENTER);
add(controlPanel_2001, BorderLayout.SOUTH);
add(scrollPane_2001, BorderLayout.EAST);

// Event Set Array
setButton_2001.addActionListener(e_2001 ->
setArrayFromInput_2001());

// Event Langkah Selanjutnya
stepButton_2001.addActionListener(e_2001 -> performStep_2001());

// Event Reset
resetButton_2001.addActionListener(e_2001 -> reset_2001());
}

private void setArrayFromInput_2001() {
    String text_2001 = inputField_2001.getText().trim();
    if (text_2001.isEmpty()) return;

    String[] parts_2001 = text_2001.split(",");
    array_2001 = new int[parts_2001.length];

    try {
        for (int i_2001 = 0; i_2001 < parts_2001.length; i_2001++) {
            array_2001[i_2001] =
Integer.parseInt(parts_2001[i_2001].trim());
        }
    } catch (NumberFormatException e_2001) {
        JOptionPane.showMessageDialog(this, "Masukkan hanya angka!",
            "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    labelArray_2001 = new JLabel[array_2001.length];
    panelArray_2001.removeAll();

    for (int i_2001 = 0; i_2001 < array_2001.length; i_2001++) {
        labelArray_2001[i_2001] = new
JLabel(String.valueOf(array_2001[i_2001]));
        labelArray_2001[i_2001].setFont(new Font("Arial", Font.BOLD,
24));
        labelArray_2001[i_2001].setOpaque(true);
        labelArray_2001[i_2001].setBackground(Color.WHITE);

labelArray_2001[i_2001].setBorder(BorderFactory.createLineBorder(Color.BLACK));
        labelArray_2001[i_2001].setPreferredSize(new Dimension(50,
50));

labelArray_2001[i_2001].setHorizontalAlignment(SwingConstants.CENTER);
        panelArray_2001.add(labelArray_2001[i_2001]);
    }
}

```



```

    }

    mergeQueue_2001.clear();
    generateMergeSteps_2001(0, array_2001.length - 1);

    stepButton_2001.setEnabled(true);
    stepArea_2001.setText("");
    stepCount_2001 = 1;
    isMerging_2001 = false;
    copying_2001 = false;

    panelArray_2001.revalidate();
    panelArray_2001.repaint();
}

private void generateMergeSteps_2001(int left_2001, int right_2001) {
    if (left_2001 < right_2001) {
        int mid_2001 = left_2001 + (right_2001 - left_2001) / 2;

        generateMergeSteps_2001(left_2001, mid_2001);
        generateMergeSteps_2001(mid_2001 + 1, right_2001);

        mergeQueue_2001.add(new int[] { left_2001, mid_2001, right_2001
    });
}

private void performStep_2001() {
    resetHighlights_2001();

    if (!isMerging_2001 && !mergeQueue_2001.isEmpty()) {
        int[] range_2001 = mergeQueue_2001.poll();

        left_2001 = range_2001[0];
        mid_2001 = range_2001[1];
        right_2001 = range_2001[2];

        temp_2001 = new int[right_2001 - left_2001 + 1];

        i_2001 = left_2001;
        j_2001 = mid_2001 + 1;
        k_2001 = 0;

        copying_2001 = false;
        isMerging_2001 = true;

        stepArea_2001.append(
            "Langkah " + stepCount_2001++
            + ": Mulai merge dari "
            + left_2001 + " ke "
            + right_2001 + "\n"
        );
    }
}

```

```

        return;
    }

    if (isMerging_2001 && !copying_2001) {
        if (i_2001 <= mid_2001 && j_2001 <= right_2001) {
            labelArray_2001[i_2001].setBackground(Color.CYAN);
            labelArray_2001[j_2001].setBackground(Color.CYAN);

            if (array_2001[i_2001] <= array_2001[j_2001]) {
                temp_2001[k_2001++] = array_2001[i_2001++];
            } else {
                temp_2001[k_2001++] = array_2001[j_2001++];
            }

            stepArea_2001.append(
                "Langkah " + stepCount_2001++
                + ": Bandingkan dan salin elemen\n"
            );

            return;
        } else if (i_2001 <= mid_2001) {
            temp_2001[k_2001++] = array_2001[i_2001++];

            stepArea_2001.append(
                "Langkah " + stepCount_2001++
                + ": Salin sisa kiri\n"
            );

            return;
        } else if (j_2001 <= right_2001) {
            temp_2001[k_2001++] = array_2001[j_2001++];

            stepArea_2001.append(
                "Langkah " + stepCount_2001++
                + ": Salin sisa kanan\n"
            );

            return;
        } else {
            copying_2001 = true;
            k_2001 = 0;
            return;
        }
    }

    if (copying_2001 && k_2001 < temp_2001.length) {
        array_2001[left_2001 + k_2001] = temp_2001[k_2001];
        labelArray_2001[left_2001 +
k_2001].setText(String.valueOf(temp_2001[k_2001]));
        labelArray_2001[left_2001 + k_2001].setBackground(Color.GREEN);
    }

```

```

        k_2001++;

        stepArea_2001.append("Langkah " + stepCount_2001++ + ":  
Tempelkan ke array utama\n");
        return;
    }

    if (copying_2001 && k_2001 == temp_2001.length) {
        isMerging_2001 = false;
        copying_2001 = false;
    }

    if (mergeQueue_2001.isEmpty() && !isMerging_2001) {
        stepArea_2001.append("Selesai.\n");
        stepButton_2001.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Merge Sort selesai!");
    }
}

private void resetHighlights_2001() {
    if (labelArray_2001 == null) return;

    for (JLabel label_2001 : labelArray_2001) {
        label_2001.setBackground(Color.WHITE);
    }
}

private void reset_2001() {
    inputField_2001.setText("");
    panelArray_2001.removeAll();
    panelArray_2001.revalidate();
    panelArray_2001.repaint();
    stepArea_2001.setText("");
    stepButton_2001.setEnabled(false);
    mergeQueue_2001.clear();
    isMerging_2001 = false;
    copying_2001 = false;
    stepCount_2001 = 1;
}

public static void main(String[] args_2001) {
    SwingUtilities.invokeLater(() -> {
        MergeSortGUI_2511532001 frame_2001 = new
MergeSortGUI_2511532001();
        frame_2001.setVisible(true);
    });
}
}

```

Program MergeSortGUI mengangkat kerumitan algoritma rekursif ke dalam visualisasi GUI. Program ini menggunakan struktur data Queue (mergeQueue). Fungsi generateMergeStep mengeksekusi logika pemisahan data di belakang layar dan

merekam urutan indeks pembelahannya ke dalam antrean (Queue). Saat pengguna menekan tombol "Langkah Selanjutnya", metode performStep akan mencabut perintah dari antrean tersebut dan mengeksekusi visualisasi penggabungan (merge) menggunakan warna (Cyan saat membandingkan, Green saat menyalin) agar pengguna dapat melihat proses penyusunan sub-array secara perlahan.

BAB 3

TUGAS

3.1 Program Sorting

```
package pekan8_2511532001;
import java.util.Scanner;

// kelas ADT lagu
class Lagu_2511532001 {
    String judul_2001;
    String penyanyi_2001;
    int durasi_2001;

    public Lagu_2511532001(String judul_2001, String penyanyi_2001, int
durasi_2001) {
        this.judul_2001 = judul_2001;
        this.penyanyi_2001 = penyanyi_2001;
        this.durasi_2001 = durasi_2001;
    }
}

public class Sorting_2511532001 {
    // array maksimal 20 lagu
    private Lagu_2511532001[] dataLagu_2001 = new Lagu_2511532001[20];
    private int jumlahLagu_2001 = 0;

    // method mengisi data awal (minimal 7 lagu)
    public void inputData_2001() {
        dataLagu_2001[0] = new Lagu_2511532001("Sunflower", "Rex Orange
Country", 270);
        dataLagu_2001[1] = new Lagu_2511532001("Wildfire", "HOYO-MiX",
220);
        dataLagu_2001[2] = new Lagu_2511532001("Dark Red", "Steve Lacy",
252);
        dataLagu_2001[3] = new Lagu_2511532001("Hope Is the Thing With
Feathers", "HOYO-MiX", 245);
        dataLagu_2001[4] = new Lagu_2511532001("White Night", "Jake Miller
& HOYO-MiX", 210);
        dataLagu_2001[5] = new Lagu_2511532001("Harvey", "Her's", 196);
        dataLagu_2001[6] = new Lagu_2511532001("If I Can Stop One Heart
From Breaking", "Robin", 235);
        dataLagu_2001[7] = new Lagu_2511532001("Bad Habit", "Steve Lacy",
232);
        dataLagu_2001[8] = new Lagu_2511532001("Dark Red", "Steve Lacy",
173);
        dataLagu_2001[9] = new Lagu_2511532001("Impostor Syndrome", "Sidney
Gish", 294);
        dataLagu_2001[10] = new Lagu_2511532001("Shots (feat. Lil Jon)",
"LMFAO", 222);
    }
}
```

```

        dataLagu_2001[11] = new Lagu_2511532001("Love Epiphany", "Reality
Club", 327);
        dataLagu_2001[12] = new Lagu_2511532001("2112", "Reality Club",
266);
        dataLagu_2001[13] = new Lagu_2511532001("Anything You Want",
"Reality Club", 235);
        dataLagu_2001[14] = new Lagu_2511532001("I Wish I Was Your Joke",
"Reality Club", 204);
        dataLagu_2001[15] = new Lagu_2511532001("Who Knows", "Daniel
Caesar", 180);
        dataLagu_2001[16] = new Lagu_2511532001("Best Friend", "Rex Orange
County", 264);
        dataLagu_2001[17] = new Lagu_2511532001("seasons", "wave to earth",
256);
        dataLagu_2001[18] = new Lagu_2511532001("daisy.", "wave to earth",
218);
        dataLagu_2001[19] = new Lagu_2511532001("bad", "wave to earth",
264);

        jumlahLagu_2001 = 20;
    }

    // method menampilkan data sebelum/sesudah
    public void tampilData_2001(String status_2001) {
        System.out.println(status_2001);
        for (int i_2001 = 0; i_2001 < jumlahLagu_2001; i_2001++) {
            System.out.println((i_2001 + 1) + ". " +
dataLagu_2001[i_2001].judul_2001 + " - " +
dataLagu_2001[i_2001].durasi_2001 + " detik");
        }
        System.out.println();
    }

    // shell sort
    public void shellSort_2001() {
        for (int gap_2001 = jumlahLagu_2001 / 2; gap_2001 > 0; gap_2001 /=
2) {
            for (int i_2001 = gap_2001; i_2001 < jumlahLagu_2001; i_2001++)
            {
                Lagu_2511532001 temp_2001 = dataLagu_2001[i_2001];
                int j_2001;
                for (j_2001 = i_2001; j_2001 >= gap_2001 &&
dataLagu_2001[j_2001 -
gap_2001].judul_2001.compareToIgnoreCase(temp_2001.judul_2001) > 0; j_2001
-= gap_2001) {
                    dataLagu_2001[j_2001] = dataLagu_2001[j_2001 -
gap_2001];
                }
                dataLagu_2001[j_2001] = temp_2001;
            }
        }
    }
}

```

```

// quick sort
private void swap_2001(int i_2001, int j_2001) {
    Lagu_2511532001 temp_2001 = dataLagu_2001[i_2001];
    dataLagu_2001[i_2001] = dataLagu_2001[j_2001];
    dataLagu_2001[j_2001] = temp_2001;
}

private int partition_2001(int low_2001, int high_2001) {
    int pivot_2001 = dataLagu_2001[high_2001].durasi_2001;
    int i_2001 = (low_2001 - 1);
    for (int j_2001 = low_2001; j_2001 <= high_2001 - 1; j_2001++) {
        if (dataLagu_2001[j_2001].durasi_2001 <= pivot_2001) {
            i_2001++;
            swap_2001(i_2001, j_2001);
        }
    }
    swap_2001(i_2001 + 1, high_2001);
    return (i_2001 + 1);
}

public void quickSort_2001(int low_2001, int high_2001) {
    if (low_2001 < high_2001) {
        int pi_2001 = partition_2001(low_2001, high_2001);
        quickSort_2001(low_2001, pi_2001 - 1);
        quickSort_2001(pi_2001 + 1, high_2001);
    }
}

// merge sort
private void merge_2001(int left_2001, int mid_2001, int right_2001) {
    int n1_2001 = mid_2001 - left_2001 + 1;
    int n2_2001 = right_2001 - mid_2001;

    Lagu_2511532001[] L_2001 = new Lagu_2511532001[n1_2001];
    Lagu_2511532001[] R_2001 = new Lagu_2511532001[n2_2001];

    for (int i_2001 = 0; i_2001 < n1_2001; ++i_2001)
        L_2001[i_2001] = dataLagu_2001[left_2001 + i_2001];
    for (int j_2001 = 0; j_2001 < n2_2001; ++j_2001)
        R_2001[j_2001] = dataLagu_2001[mid_2001 + 1 + j_2001];

    int i_2001 = 0, j_2001 = 0;
    int k_2001 = left_2001;

    while (i_2001 < n1_2001 && j_2001 < n2_2001) {
        // bandingkan judul secara alfabetis (A-Z) mengabaikan kapital
        if
(L_2001[i_2001].judul_2001.compareToIgnoreCase(R_2001[j_2001].judul_2001)
<= 0) {
            dataLagu_2001[k_2001] = L_2001[i_2001];
            i_2001++;
        } else {
            dataLagu_2001[k_2001] = R_2001[j_2001];

```

```

        j_2001++;
    }
    k_2001++;
}

while (i_2001 < n1_2001) {
    dataLagu_2001[k_2001] = L_2001[i_2001];
    i_2001++;
    k_2001++;
}

while (j_2001 < n2_2001) {
    dataLagu_2001[k_2001] = R_2001[j_2001];
    j_2001++;
    k_2001++;
}
}

public void mergeSort_2001(int left_2001, int right_2001) {
    if (left_2001 < right_2001) {
        int mid_2001 = left_2001 + (right_2001 - left_2001) / 2;

        mergeSort_2001(left_2001, mid_2001);
        mergeSort_2001(mid_2001 + 1, right_2001);

        merge_2001(left_2001, mid_2001, right_2001);
    }
}

public static void main(String[] args) {
    Scanner sc_2001 = new Scanner(System.in);
    Sorting_2511532001 playlist_2001 = new Sorting_2511532001();

    System.out.println("Sorting Playlist NIM: 2511532001");
    System.out.print("Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): ");
    int pilihan_2001 = sc_2001.nextInt();

    // inisialisasi data mentah
    playlist_2001.inputData_2001();
    playlist_2001.tampilData_2001("Data Sebelum Sorting:");

    switch (pilihan_2001) {
        case 1:
            playlist_2001.shellSort_2001();
            playlist_2001.tampilData_2001("Data Setelah Shell Sort (Judul A-Z):");
            break;
        case 2:
            playlist_2001.quickSort_2001(0,
playlist_2001.jumlahLagu_2001 - 1);
            playlist_2001.tampilData_2001("Data Setelah Quick Sort (Durasi Asc):");
            break;
    }
}

```



```

        case 3:
            playlist_2001.mergeSort_2001(0,
playlist_2001.jumlahLagu_2001 - 1);
            playlist_2001.tampilData_2001("Data Setelah Merge Sort
(Judul A-Z):");
            break;
        default:
            System.out.println("Pilihan tidak valid!");
    }
    sc_2001.close();
}
}

```

Program Sorting ini dirancang untuk mendukung tiga algoritma pengurutan dalam satu kali eksekusi. Data playlist lagu disimpan dalam struktur array statis berkapasitas 20 elemen, yang masing-masing elemennya merupakan objek Lagu.

Alur program dikendalikan melalui menu berbasis Scanner dengan struktur percabangan switch-case untuk menentukan metode pengurutan yang dipilih oleh pengguna. Metode shellSort digunakan untuk mengurutkan judul lagu secara alfabetis dengan pendekatan diminishing increment (gap), yaitu membandingkan elemen dalam jarak tertentu yang semakin mengecil.

Selanjutnya, metode quickSort berfungsi mengurutkan durasi lagu secara ascending dengan teknik partisi in-place, menggunakan elemen terakhir sebagai pivot. Sementara itu, metode mergeSort menerapkan konsep Divide and Conquer dengan membagi array menjadi bagian kecil, lalu menggabungkannya kembali untuk menghasilkan urutan judul lagu secara leksikografis menggunakan fungsi komparasi teks bawaan Java.

3.2 Alasan Implementasi Algoritma

Ketiga algoritma tersebut diimplementasikan secara lengkap dalam satu program untuk menunjukkan kemampuan serta karakteristik unik dari masing-masing metode pengurutan tingkat lanjut. Shell Sort dimanfaatkan untuk mengoptimalkan pergeseran data secara alfabetis pada judul lagu tanpa perlu alokasi memori tambahan.

Quick Sort digunakan sebagai metode yang efisien untuk menangani perbandingan data numerik, khususnya pada atribut durasi lagu. Sementara itu, Merge Sort berperan sebagai penyeimbang karena memiliki sifat stabil, sehingga mampu menjaga konsistensi hasil pengurutan teks dalam berbagai kondisi.

BAB 4

KESIMPULAN

4.1 Ringkasan Hasil Praktikum

Berdasarkan praktikum yang dilakukan, dapat disimpulkan bahwa algoritma pengurutan advanced memiliki efisiensi waktu yang jauh lebih baik dibandingkan algoritma fundamental. Shell Sort berhasil mempercepat kinerja Insertion Sort dengan mengurangi jumlah pergeseran data dengan teknik diminishing increment (gap). Quick Sort, dengan optimasi Median-of-Three, sebagai salah satu algoritma tercepat untuk pengurutan in-place tanpa memori tambahan yang besar. Di sisi lain, Merge Sort memberikan jaminan performa konsisten dan stabil pada berbagai kondisi data, meskipun menuntut kompensasi berupa alokasi memori tambahan (temporary array) saat proses penggabungan. Implementasi visualisasi GUI menunjukkan bahwa logika algoritma rekursif yang berjalan secara instan dapat diurai ke dalam bentuk antrian (Queue) agar dapat diobservasi oleh pengguna.

DAFTAR PUSTAKA

- [1] GeeksforGeeks, "Sorting Algorithms in Java". [Daring]. Tersedia pada:
<https://www.geeksforgeeks.org/sorting-algorithms/>. [Diakses: 3 Juni 2026].